Лабораторна робота №2

Інструменти конфігурування мережі в Linux системах

Мета роботи — ознайомитись з мережевими інструментами в Linux, які використовуються для відладки & налаштування мережі.

ТЕОРЕТИЧНІ ВІДОМОСТІ

Навіщо потрібні такі інструменти?

Робота мережі для пересічного користувача виглядає в деякому роді магією, оскільки багато операцій автоматичні і не вимагають прямої взаємодії. Linux дає можливість тонкої конфігурації. Для цього потрібно внести необхідні зміни в певні файли, але це незручно. Використання спеціальних консольних команд дає можливість спростити та виключити людський фактор.

Що таке ifconfig?

Дана команда дозволяє подивитися список наявних інтерфейсів. Її виконання без опцій виведе інформацію тільки про активні інтерфейси.

```
┌──(vnikitin㉿vnikitin)-[~]
└─$ ifconfig
br-b4c8a56664cf: flags=4163<UP,BROADCAST,RUNNING,MULTICAST>  mtu 1500
        inet 172.20.0.1  netmask 255.255.255.0  broadcast 172.20.0.255
        inet6 fe80::42:caff:fe90:4e3  prefixlen 64  scopeid 0x20<link>
        ether 02:42:ca:90:04:e3  txqueuelen 0  (Ethernet)
        RX packets 13616163  bytes 4780369830 (4.4 GiB)
        RX errors 0  dropped 0  overruns 0  frame 0
        TX packets 13693549  bytes 5073907260 (4.7 GiB)
        TX errors 0  dropped 0 overruns 0  carrier 0  collisions 0

docker0: flags=4099<UP,BROADCAST,MULTICAST>  mtu 1500
        inet 172.17.0.1  netmask 255.255.0.0  broadcast 172.17.255.255
        ether 02:42:66:3e:ed:3b  txqueuelen 0  (Ethernet)
        RX packets 0  bytes 0 (0.0 B)
        RX errors 0  dropped 0  overruns 0  frame 0
        TX packets 0  bytes 0 (0.0 B)
        TX errors 0  dropped 0 overruns 0  carrier 0  collisions 0
```

```
┌──(vnikitin㉿vnikitin)-[~]
└─$ ifconfig eth0
eth0: flags=195<UP,BROADCAST,RUNNING,NOARP>  mtu 1500
        inet 10.1.1.125  netmask 255.0.0.0  broadcast 10.255.255.255
        inet6 fe80::b855:d4ff:fe4c:512b  prefixlen 64  scopeid 0x20<link>
        ether ba:55:d4:4c:51:2b  txqueuelen 1000  (Ethernet)
        RX packets 0  bytes 0 (0.0 B)
        RX errors 0  dropped 0  overruns 0  frame 0
        TX packets 40  bytes 2800 (2.7 KiB)
        TX errors 0  dropped 0 overruns 0  carrier 0  collisions 0
```

```
┌──(vnikitin㉿vnikitin)-[~]
└─$ ifconfig eth0
eth0: flags=195<UP,BROADCAST,RUNNING,NOARP>  mtu 1500
        inet 10.1.1.15  netmask 255.0.0.0  broadcast 10.255.255.255
        inet6 fe80::b855:d4ff:fe4c:512b  prefixlen 64  scopeid 0x20<link>
        ether ba:55:d4:4c:51:2b  txqueuelen 1000  (Ethernet)
        RX packets 0  bytes 0 (0.0 B)
        RX errors 0  dropped 0  overruns 0  frame 0
        TX packets 40  bytes 2800 (2.7 KiB)
        TX errors 0  dropped 0 overruns 0  carrier 0  collisions 0

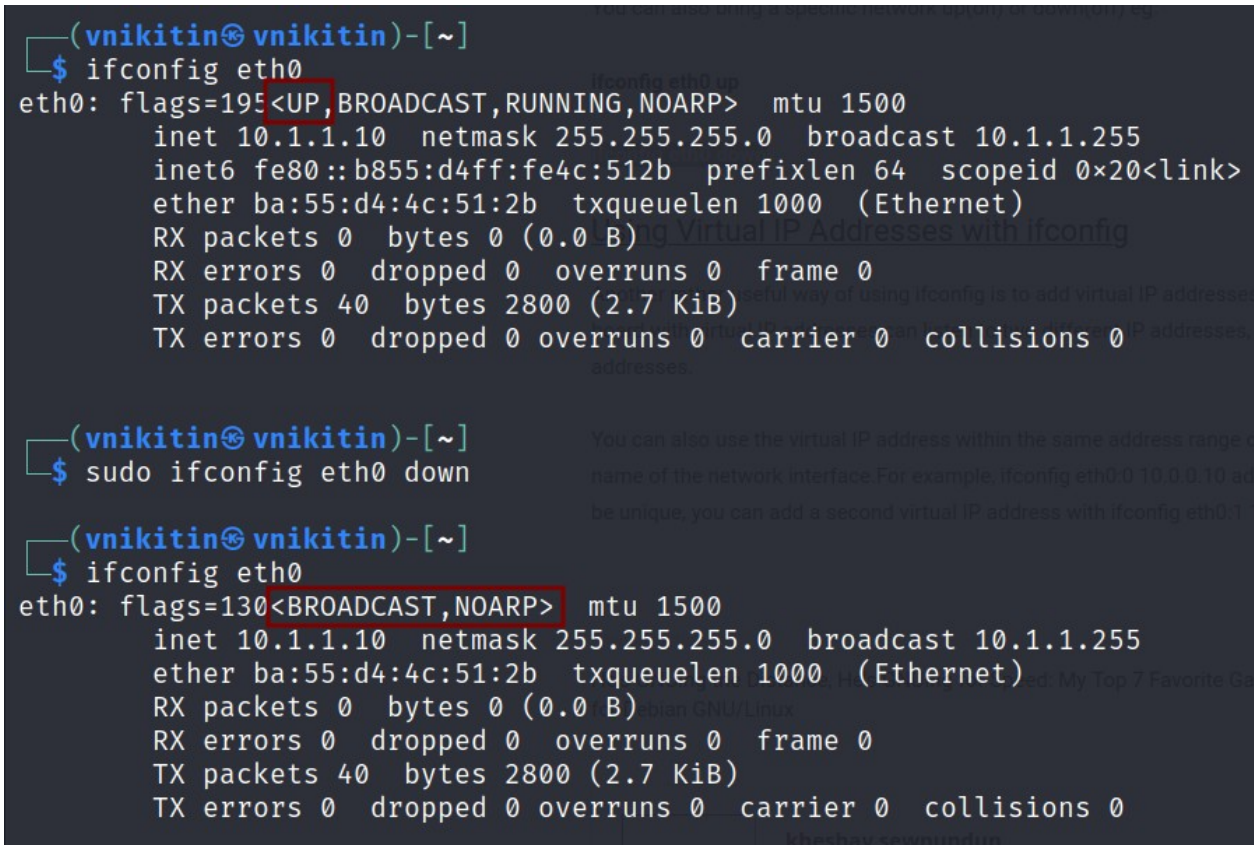

┌──(vnikitin㉿vnikitin)-[~]
└─$ sudo ifconfig eth0 10.1.1.10 up

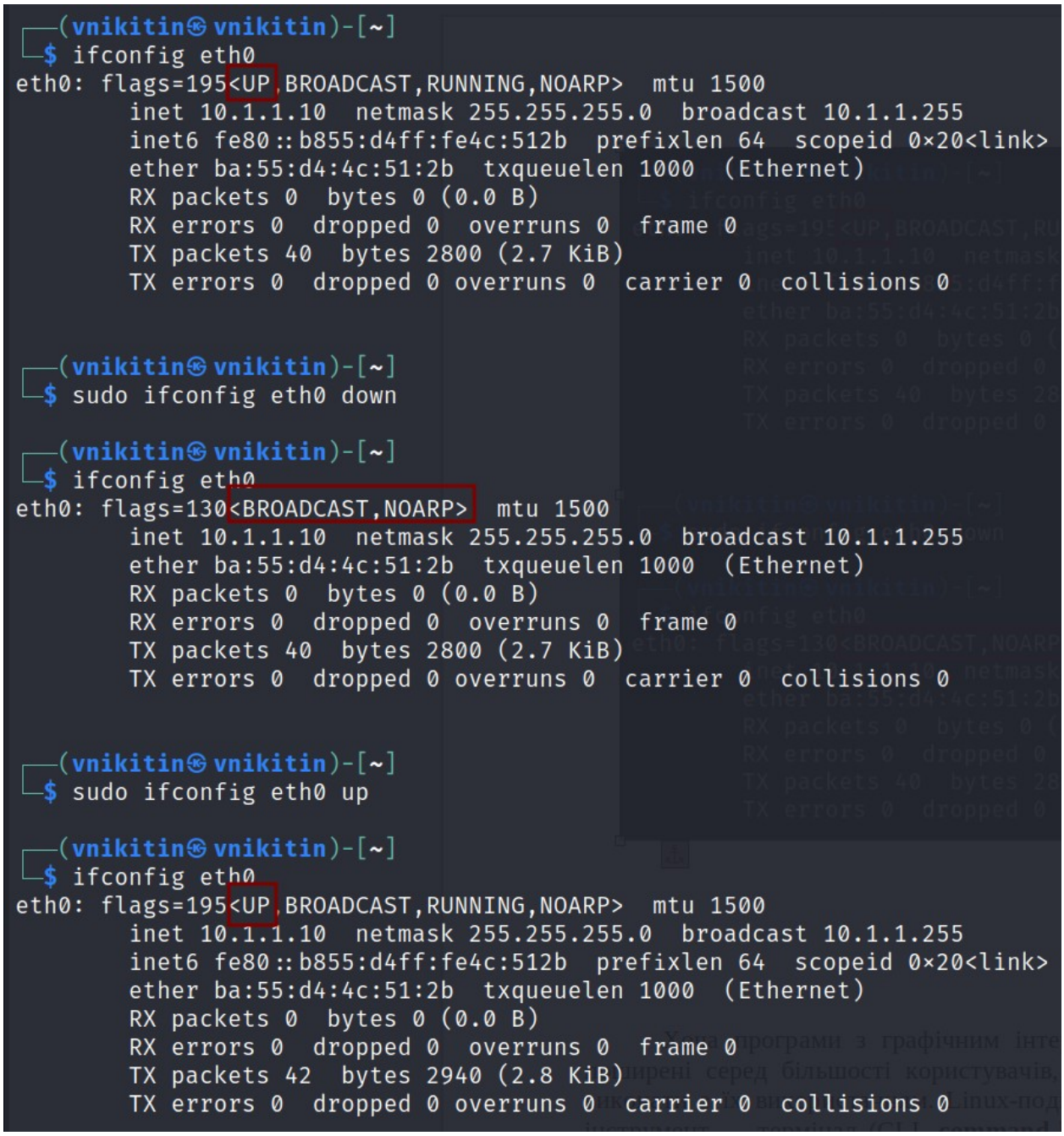
┌──(vnikitin㉿vnikitin)-[~]
└─$ ifconfig eth0
eth0: flags=195<UP,BROADCAST,RUNNING,NOARP>  mtu 1500
        inet 10.1.1.10  netmask 255.0.0.0  broadcast 10.255.255.255
        inet6 fe80::b855:d4ff:fe4c:512b  prefixlen 64  scopeid 0x20<link>
        ether ba:55:d4:4c:51:2b  txqueuelen 1000  (Ethernet)
        RX packets 0  bytes 0 (0.0 B)
        RX errors 0  dropped 0  overruns 0  frame 0
        TX packets 40  bytes 2800 (2.7 KiB)
        TX errors 0  dropped 0 overruns 0  carrier 0  collisions 0
```

```
┌──(vnikitin㉿vnikitin)-[~]
└─$ ifconfig eth0
eth0: flags=195<UP,BROADCAST,RUNNING,NOARP>  mtu 1500
        inet 10.1.1.10  netmask 255.0.0.0  broadcast 10.255.255.255
        inet6 fe80::b855:d4ff:fe4c:512b  prefixlen 64  scopeid 0x20<link>
        ether ba:55:d4:4c:51:2b  txqueuelen 1000  (Ethernet)
        RX packets 0  bytes 0 (0.0 B)
        RX errors 0  dropped 0  overruns 0  frame 0
        TX packets 40  bytes 2800 (2.7 KiB)
        TX errors 0  dropped 0 overruns 0  carrier 0  collisions 0


┌──(vnikitin㉿vnikitin)-[~]
└─$ sudo ifconfig eth0 10.1.1.10 netmask 255.255.255.0 up

┌──(vnikitin㉿vnikitin)-[~]
└─$ ifconfig eth0
eth0: flags=195<UP,BROADCAST,RUNNING,NOARP>  mtu 1500
        inet 10.1.1.10  netmask 255.255.255.0  broadcast 10.1.1.255
        inet6 fe80::b855:d4ff:fe4c:512b  prefixlen 64  scopeid 0x20<link>
        ether ba:55:d4:4c:51:2b  txqueuelen 1000  (Ethernet)
        RX packets 0  bytes 0 (0.0 B)
        RX errors 0  dropped 0  overruns 0  frame 0
        TX packets 40  bytes 2800 (2.7 KiB)
        TX errors 0  dropped 0 overruns 0  carrier 0  collisions 0
```


(vnikitin㉿vnikitin)-[~]
$ ifconfig eth0
eth0: flags=195<UP,BROADCAST,RUNNING,NOARP> mtu 1500
inet 10.1.1.10 netmask 255.255.255.0 broadcast 10.1.1.255
inet6 fe80::b855:d4ff:fe4c:512b prefixlen 64 scopeid 0×20<link>
ether ba:55:d4:4c:51:2b txqueuelen 1000 (Ethernet)
RX packets 0 bytes 0 (0.0 B)
RX errors 0 dropped 0 overruns 0 frame 0
TX packets 40 bytes 2800 (2.7 KiB)
TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0
(vnikitin㉿vnikitin)-[~]
$ sudo ifconfig eth0 down
(vnikitin㉿vnikitin)-[~]
$ ifconfig eth0
eth0: flags=130<BROADCAST,NOARP> mtu 1500
inet 10.1.1.10 netmask 255.255.255.0 broadcast 10.1.1.255
ether ba:55:d4:4c:51:2b txqueuelen 1000 (Ethernet)
RX packets 0 bytes 0 (0.0 B)
RX errors 0 dropped 0 overruns 0 frame 0
TX packets 40 bytes 2800 (2.7 KiB)
TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0



Що таке ethtool?

Ethtool — ще одна утиліта командного рядка для відображення та зміни параметрів контролерів мережевого інтерфейсу (NIC) і програмного забезпечення драйверів пристроїв. Цей інструмент також корисний для ідентифікації та діагностики пристроїв Ethernet, оновлення мікропрограми у флеш-пам’яті тощо.

```
root@host1:/# ethtool eth0
Settings for eth0:
        Supported ports: [ ]
        Supported link modes:   Not reported
        Supported pause frame use: No
        Supports auto-negotiation: No
        Supported FEC modes: Not reported
        Advertised link modes:  Not reported
        Advertised pause frame use: No
        Advertised auto-negotiation: No
        Advertised FEC modes: Not reported
        Speed: 10000Mb/s
        Duplex: Full
        Port: Twisted Pair
        PHYAD: 0
        Transceiver: internal
        Auto-negotiation: off
        MDI-X: Unknown
```

Що таке ip?

Команда IP є свого роду заміною команди ifconfig. Ця команда використовується для виконання різних завдань адміністрування мережі. Ця команда використовується для відображення або керування маршрутизацією, пристроями, мережевими пристроями та тунелями. Хоча вона схожа на команду ifconfig, але надто потужна, включаючи більше функцій і засобів.

```
┌──(vnikitin㉿vnikitin)-[~]
└─$ ip addr show
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
       valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host
       valid_lft forever preferred_lft forever
2: wlan0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc noqueue state UP group default qlen 1000
    link/ether [redacted]:5a:cf:26 brd ff:ff:ff:ff:ff:ff
    inet 192.168.0.103/24 brd 192.168.0.255 scope global dynamic noprefixroute wlan0
       valid_lft 4532sec preferred_lft 4532sec
    inet6 fe80::5027:1b00:4fe1:5efa/64 scope link noprefixroute
       valid_lft forever preferred_lft forever
3: docker0: <NO-CARRIER,BROADCAST,MULTICAST,UP> mtu 1500 qdisc noqueue state DOWN group default
    link/ether 02:42:66:3e:ed:3b brd ff:ff:ff:ff:ff:ff
    inet 172.17.0.1/16 brd 172.17.255.255 scope global docker0
       valid_lft forever preferred_lft forever
```

```
┌──(vnikitin㉿vnikitin)-[~]
└─$ ip addr show eth0
15: eth0: <BROADCAST,NOARP,UP,LOWER_UP> mtu 1500 qdisc noqu
    link/ether ba:55:d4:4c:51:2b brd ff:ff:ff:ff:ff:ff
    inet 10.1.1.10/24 brd 10.1.1.255 scope global eth0
       valid_lft forever preferred_lft forever
    inet6 fe80::b855:d4ff:fe4c:512b/64 scope link
       valid_lft forever preferred_lft forever
```

```
┌──(vnikitin㉿vnikitin)-[~]
└─$ sudo ip addr add 195.162.54.2/24 dev eth0

┌──(vnikitin㉿vnikitin)-[~]
└─$ ip addr show eth0
15: eth0: <BROADCAST,NOARP,UP,LOWER_UP> mtu 1500 qdisc n
    link/ether ba:55:d4:4c:51:2b brd ff:ff:ff:ff:ff:ff
    inet 10.1.1.10/24 brd 10.1.1.255 scope global eth0
       valid_lft forever preferred_lft forever
    inet 195.162.54.2/24 scope global eth0
       valid_lft forever preferred_lft forever
    inet6 fe80::b855:d4ff:fe4c:512b/64 scope link
       valid_lft forever preferred_lft forever

┌──(vnikitin㉿vnikitin)-[~]
└─$ sudo ip addr del 195.162.54.2/24 dev eth0

┌──(vnikitin㉿vnikitin)-[~]
└─$ ip addr show eth0
15: eth0: <BROADCAST,NOARP,UP,LOWER_UP> mtu 1500 qdisc n
    link/ether ba:55:d4:4c:51:2b brd ff:ff:ff:ff:ff:ff
    inet 10.1.1.10/24 brd 10.1.1.255 scope global eth0
       valid_lft forever preferred_lft forever
    inet6 fe80::b855:d4ff:fe4c:512b/64 scope link
       valid_lft forever preferred_lft forever
```

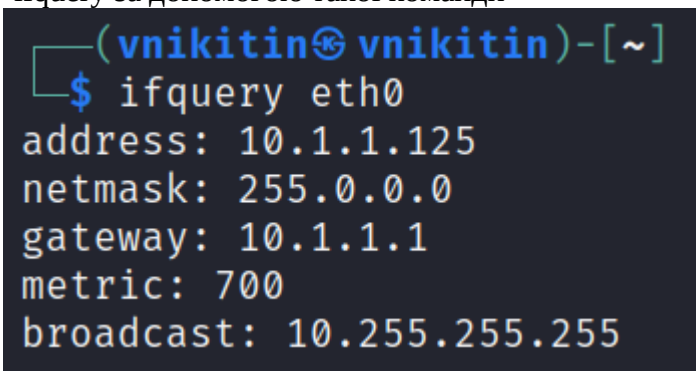
```
┌──(vnikitin㉿vnikitin)-[~]
└─$ ip neigh
172.20.0.12 dev br-b4c8a56664cf lladdr 02:42:ac:14:00:0c REACHABLE
192.168.0.1 dev wlan0 lladdr 00:31:92:7d:1a:9a REACHABLE
172.20.0.11 dev br-b4c8a56664cf lladdr 02:42:ac:14:00:0b REACHABLE
172.20.0.13 dev br-b4c8a56664cf lladdr 02:42:ac:14:00:0d REACHABLE
```

Що таке ifup, ifdown, ifquery?

Команда ifup загалом включає мережевий інтерфейс, що дозволяє користувачеві передавати й отримувати дані.

Команда ifdown відключає мережевий інтерфейс, що не дозволяє користувачеві передавати та отримувати дані.

Щоб відобразити інформацію про конфігурацію мережевого інтерфейсу, ми використовуємо команду ifquery за допомогою такої команди




```
(vnikitin㉿vnikitin)-[~]
$ sudo ifdown -v eth0
ifdown: parsing file /etc/network/interfaces.d/eth0
ifdown: configuring interface eth0=eth0 (inet)
run-parts --verbose /etc/network/if-down.d
run-parts: executing /etc/network/if-down.d/bridge
run-parts: executing /etc/network/if-down.d/openvpn
run-parts: executing /etc/network/if-down.d/wpasupplicant
 ip route del default via 10.1.1.1 metric 700 dev eth0 2>&1 1>/dev/null || true
RTNETLINK answers: No such process
ip addr del 10.1.1.125/255.0.0.0 broadcast 10.255.255.255       dev eth0 label eth0
RTNETLINK answers: Cannot assign requested address
ip -4 addr flush dev eth0
ip link set dev eth0 down
run-parts --verbose /etc/network/if-post-down.d
run-parts: executing /etc/network/if-post-down.d/bridge
run-parts: executing /etc/network/if-post-down.d/ifenslave
+ [ inet = meta ]
+ BOND_PARAMS=/sys/class/net/eth0/bonding
+ [ -f /sys/class/net/eth0/master/bonding/slaves ]
+ [ ! -f /sys/class/net/eth0/bonding/slaves ]
+ exit
run-parts: executing /etc/network/if-post-down.d/macchanger
run-parts: executing /etc/network/if-post-down.d/vlan
run-parts: executing /etc/network/if-post-down.d/wireless-tools
run-parts: executing /etc/network/if-post-down.d/wpasupplicant
```

Що таке ping?

Команда Ping (Packet INternet Groper) — це утиліта адміністрування комп’ютерної мережі для перевірки доступності між двома вузлами в локальній мережі (LAN) або глобальній мережі (WAN). Для зв’язку з вузлами в мережі ping надсилає пакети ехо-запиту протоколу ICMP (Internet Control Message Protocol) на цільовий хост і очікує ехо-відповіді ICMP, яка показує, що пінг пройшов успішно.

ICMP надає спільний доступ до інформації про помилки та оновлення статусу пакетів даних. Це корисно для виявлення та усунення помилок мережі. ICMP повідомляє інформацію про пакети, які були відкинуті або зникли під час передавання, проблеми з підключенням до мережі та пакети, перенаправлені на інші маршрутизатори.

Але якщо передається занадто багато повідомлень ICMP, ехо-відповіді ICMP

переповнюють сервери в мережі, і вони “падають”. Це створює відмову в обслуговуванні та може призвести до зупинки діяльності організації.

ICMP протокол може використовуватись для DoS-атаки, що виконується зловмисником, який неодноразово надсилає пакети запитів ICMP на мережевий сервер.

```
┌──(vnikitin㉿vnikitin)-[~]
└─$ ping -c 3 -I wlan0 -l 2 -s 64 8.8.8.8
PING 8.8.8.8 (8.8.8.8) from 192.168.0.103 wlan0: 64(92) bytes of data.
72 bytes from 8.8.8.8: icmp_seq=1 ttl=115 time=25.7 ms
72 bytes from 8.8.8.8: icmp_seq=2 ttl=115 time=35.5 ms
72 bytes from 8.8.8.8: icmp_seq=3 ttl=115 time=36.1 ms

── 8.8.8.8 ping statistics ──
3 packets transmitted, 3 received, 0% packet loss, time 1002ms
rtt min/avg/max/mdev = 25.721/32.443/36.080/4.758 ms, pipe 2

┌──(vnikitin㉿vnikitin)-[~]
└─$ sudo ping -f -c 3 -I wlan0 -l 2 -s 64 8.8.8.8
PING 8.8.8.8 (8.8.8.8) from 192.168.0.103 wlan0: 64(92) bytes of data.

── 8.8.8.8 ping statistics ──
3 packets transmitted, 3 received, 0% packet loss, time 12ms
rtt min/avg/max/mdev = 27.365/32.461/39.147/4.939 ms, pipe 3, ipg/ewma 6.207/31.338 ms
```

Що таке netstat?

netstat представляє собою утиліту, яка використовується для отримання корисної інформації щодо мережевих підключень, таблиць маршрутизації, статистики інтерфейсу. Це також дуже корисно для пошуку проблем у мережі та їх усунення, а також визначає обсяг трафіку в мережі.

Оскільки вона є інструментом налагодження мережевих служб, вона використовується для перевірки того, які програми прослуховують які порти.

```
┌──(vnikitin㉿vnikitin)-[~]
└─$ sudo netstat -tnlp
Active Internet connections (only servers)
Proto Recv-Q Send-Q Local Address           Foreign Address         State       PID/Program name
tcp        0      0 0.0.0.0:22              0.0.0.0:*               LISTEN      755/sshd: /usr/sbin
tcp        0      0 0.0.0.0:9081            0.0.0.0:*               LISTEN      1989/docker-proxy
tcp        0      0 127.0.0.1:35941         0.0.0.0:*               LISTEN      772/containerd
tcp        0      0 0.0.0.0:30001           0.0.0.0:*               LISTEN      1299/docker-proxy
tcp        0      0 0.0.0.0:30002           0.0.0.0:*               LISTEN      1269/docker-proxy
tcp        0      0 0.0.0.0:30003           0.0.0.0:*               LISTEN      1415/docker-proxy
tcp6       0      0 :::22                   :::*                    LISTEN      755/sshd: /usr/sbin
tcp6       0      0 127.0.0.1:63342         :::*                    LISTEN      6408/java
tcp6       0      0 ::1:52829               :::*                    LISTEN      2606/./jetbrains-to
tcp6       0      0 :::9081                 :::*                    LISTEN      1995/docker-proxy
tcp6       0      0 :::30001                :::*                    LISTEN      1305/docker-proxy
tcp6       0      0 :::30002                :::*                    LISTEN      1275/docker-proxy
tcp6       0      0 :::30003                :::*                    LISTEN      1422/docker-proxy
tcp6       0      0 127.0.0.1:6942          :::*                    LISTEN      6408/java
```

```
┌──(vnikitin㉿vnikitin)-[~]
└─$ netstat -r
Kernel IP routing table
Destination     Gateway         Genmask         Flags   MSS Window  irtt Iface
default         192.168.0.1     0.0.0.0         UG        0 0          0 wlan0
172.17.0.0      0.0.0.0         255.255.0.0     U         0 0          0 docker0
172.20.0.0      0.0.0.0         255.255.255.0   U         0 0          0 br-b4c8a56664cf
192.168.0.0     0.0.0.0         255.255.255.0   U         0 0          0 wlan0
```

```
┌──(vnikitin㉿vnikitin)-[~]
└─$ netstat -a
Active Internet connections (servers and established)
Proto Recv-Q Send-Q Local Address           Foreign Address         State
tcp        0      0 0.0.0.0:ssh             0.0.0.0:*               LISTEN
tcp        0      0 0.0.0.0:9081            0.0.0.0:*               LISTEN
tcp        0      0 localhost2:35941        0.0.0.0:*               LISTEN
tcp        0      0 0.0.0.0:30001           0.0.0.0:*               LISTEN
tcp        0      0 0.0.0.0:30002           0.0.0.0:*               LISTEN
tcp        0      0 0.0.0.0:30003           0.0.0.0:*               LISTEN
tcp        0      0 192.168.0.108:55830     52.113.199.174:https    ESTABLISHED
tcp        0      0 192.168.0.108:46138     ec2-54-84-92-154.:https TIME_WAIT
tcp        0      0 192.168.0.108:40562     178.128.135.204:https   ESTABLISHED
tcp        0      0 192.168.0.108:36654     ec2-34-198-139-16:https ESTABLISHED
tcp        0      0 192.168.0.108:40796     52.98.229.114:https     ESTABLISHED
tcp        0      0 192.168.0.108:45927     52.114.74.221:https     ESTABLISHED
tcp        0      0 192.168.0.108:45002     52.113.195.132:https    ESTABLISHED
tcp        0      0 192.168.0.108:44676     185.192.249.18:https    TIME_WAIT
tcp        0      0 192.168.0.108:51632     waw02s22-in-f10.1:https TIME_WAIT
tcp        0      0 192.168.0.108:40244     waw40s02-in-f1.1e:https ESTABLISHED
```

Що таке nc?

nc (NetCat) — це чудовий інструмент безпеки або інструмент моніторингу мережі, який часто називають швейцарським армійським ножем мережевих інструментів.

Він використовується для читання та запису в мережеві підключення за допомогою TCP або UDP. Він має деякі чудові функції, зокрема сканування портів, передачу файлів і прослуховування портів, і його можуть використовувати хакери як бекдор.

```
┌──(vnikitin㉿vnikitin)-[~]
└─$ netcat -z -v 127.0.0.1 40578
localhost2 [127.0.0.1] 40578 (?) : Connection refused

┌──(vnikitin㉿vnikitin)-[~]
└─$ netcat -z -v 127.0.0.1 22
localhost2 [127.0.0.1] 22 (ssh) open
```

```
┌──(vnikitin㉿vnikitin)-[~]
└─$ printf "GET /nc.1 HTTPs/1.1\r\nHost: kpi.ua\r\n\r\n" | nc kpi.ua 443
HTTP/1.1 400 Bad Request
Server: nginx/1.18.0
Date: Thu, 10 Aug 2023 12:56:17 GMT
Content-Type: text/html
Content-Length: 157
Connection: close

<html>
<head><title>400 Bad Request</title></head>
<body>
<center><h1>400 Bad Request</h1></center>
<hr><center>nginx/1.18.0</center>
</body>
</html>
```

Що таке nmap?

Nmap (Network Mapper) — це потужний мережевий сканер із відкритим вихідним кодом. Для виявлення хостів і служб у комп'ютерній мережі Nmap використовується шляхом надсилання пакетів. Функції Nmap включають сканування портів, визначення MAC-адрес, визначення версії, виявлення ОС, інвентаризацію мережі, відображення мережі тощо.

```
┌──(vnikitin㉿vnikitin)-[~]
└─$ nmap 127.0.0.1
Starting Nmap 7.92 ( https://nmap.org ) at 2023-08-10 19:45 EEST
Nmap scan report for localhost2 (127.0.0.1)
Host is up (0.000071s latency).
Not shown: 998 closed tcp ports (conn-refused)
PORT     STATE SERVICE
22/tcp   open  ssh
9081/tcp open  cisco-aqos

Nmap done: 1 IP address (1 host up) scanned in 0.07 seconds
```

Що таке iptables?

iptables — це утиліта фільтрації пакетів, подібна до брандмауера, яка використовується системним адміністратором для налаштування, підтримки та перевірки правил фільтрації IP-пакетів брандмауера ядра Linux. Брандмауер Linux (Netfilter) налаштовується та керується iptables. Iptables дозволяють додавати, видаляти або змінювати правила фільтрації пакетів у існуючих правилах фільтрації пакетів.

```
┌──(vnikitin㉿vnikitin)-[~]
└─$ sudo iptables -L
Chain INPUT (policy ACCEPT)
target     prot opt source               destination

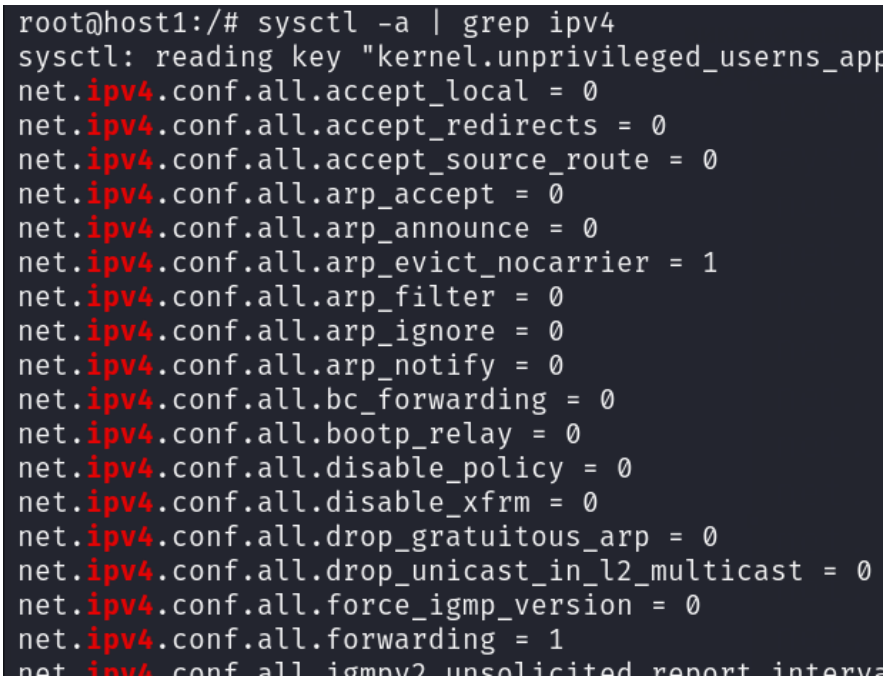
Chain FORWARD (policy DROP)
target     prot opt source               destination
DOCKER-USER  all  --  anywhere             anywhere
DOCKER-ISOLATION-STAGE-1  all  --  anywhere             anywhere
ACCEPT     all  --  anywhere             anywhere             ctstate RELATED,ESTABLISHED
DOCKER     all  --  anywhere             anywhere
ACCEPT     all  --  anywhere             anywhere
ACCEPT     all  --  anywhere             anywhere
ACCEPT     all  --  anywhere             anywhere             ctstate RELATED,ESTABLISHED
DOCKER     all  --  anywhere             anywhere
ACCEPT     all  --  anywhere             anywhere
ACCEPT     all  --  anywhere             anywhere

Chain OUTPUT (policy ACCEPT)
```

Що таке sysctl?

Команда sysctl зчитує інформацію з каталогу /proc/sys. /proc/sys — це віртуальний каталог, який містить файлові об'єкти, які можна використовувати для перегляду та встановлення поточних параметрів ядра.

Ви також можете переглянути значення параметра, відобразивши вміст відповідного файлу. Єдина відмінність полягає в тому, як представлено файл. Наприклад, і sysctl vm.swappiness, і cat /proc/sys/vm/swappiness дадуть однаковий результат. Під час використання sysctl косі риски каталогу замінюються крапками, а частина proc.sys передбачається.



Що таке tcpdump?

Tcpdump — це утиліта командного рядка, яка дозволяє отримувати та аналізувати мережевий трафік, що проходить через хост. Він часто використовується для вирішення проблем з мережею, а також як інструмент безпеки.

Потужний і універсальний інструмент, який містить багато параметрів і фільтрів, tcpdump можна використовувати в різних випадках. Оскільки це інструмент командного рядка, він ідеально підходить для запуску на віддалених серверах або пристроях, для яких графічний інтерфейс недоступний, щоб збирати дані, які можна проаналізувати пізніше. Його також можна запускати у фоновому режимі або як заплановане завдання за допомогою таких інструментів, як cron.

ПІДГОТОВКА ДО ВИКОНАННЯ ЛАБОРАТОРНОЇ РОБОТИ №2

Перед виконанням лабораторної роботи рекомендується ознайомитись з відповідним розділом лекційного матеріалу курсу та засвоїти теоретичний матеріал.

ВИМОГИ ДО ОФОРМЛЕННЯ ЗВІТУ

Звіт з лабораторної роботи обов’язково повинен містити наступну інформацію:
- назва комп’ютерного практикуму;
- мета роботи;
- постановка завдання і алгоритм його розв’язання;
- відповіді на завдання у текстовому форматі та графічними зображеннями за необхідності.

Завдання на лабораторну роботу

Завдання 2.1. Підготувати середовище

Завдання 2.1.1. Створити Dockerfile

```
┌──(vnikitin㉿vnikitin)-[~/Documents/KPI/Викладання/МУтаП/lab2]
└─$ cat Dockerfile
FROM ubuntu:20.04
ARG DEBIAN_FRONTEND=noninteractive
RUN apt-get update && apt-get install -y vim iproute2 iputils-ping \
                                         nmap net-tools ethtool ifupdown \
                                         netcat iptables tcpdump bridge-utils \
                                         arping
```

Завдання 2.1.2. Створити docker-compose.yml

```
┌──(vnikitin㉿vnikitin)-[~/Documents/KPI/Викладання/МУтаП/lab2]
└─$ cat docker-compose.yml
services:
  host1:
    build:
      context: .
      dockerfile: Dockerfile
    image: mutap_ubuntu
    hostname: host1
    container_name: host1
    tty: true
    networks:
      default:
        ipv4_address: 10.10.0.2
    privileged: true
  host2:
    build:
      context: .
      dockerfile: Dockerfile
    image: mutap_ubuntu
    hostname: host2
    container_name: host2
    tty: true
    networks:
      default:
        ipv4_address: 10.10.0.3
    privileged: true

networks:
  default:
    driver: bridge
    ipam:
      config:
        - subnet: 10.10.0.0/24
```

Завдання 2.1.3. Перевірити, що контейнери працюють

```
┌──(vnikitin㉿vnikitin)-[~/Documents/KPI/Викладання/МУтаП/lab2]
└─$ docker-compose ps
Name     Command    State   Ports
------------------------------------
host1   /bin/bash   Up
host2   /bin/bash   Up
```

Завдання 2.1.4. Підключитись до контейнеру та спробувати пропінгувати host2

```
┌──(vnikitin㉿vnikitin)-[~/Documents/КРІ/Викладання/МУтаП/lab2]
└─$ docker container exec -ti host1 bash
root@host1:/# ping host2
PING host2 (10.10.0.3) 56(84) bytes of data.
64 bytes from host2.lab2_default (10.10.0.3): icmp_seq=1 ttl=64 time=0.171 ms
64 bytes from host2.lab2_default (10.10.0.3): icmp_seq=2 ttl=64 time=0.106 ms
64 bytes from host2.lab2_default (10.10.0.3): icmp_seq=3 ttl=64 time=0.100 ms
^C
── host2 ping statistics ──
3 packets transmitted, 3 received, 0% packet loss, time 2044ms
rtt min/avg/max/mdev = 0.100/0.125/0.171/0.032 ms
root@host1:/# exit
exit
```

Завдання 2.2. Налаштувати вручну зв’язок між host1 та host2

2.2.1. Налаштувати брідж та інтерфейси на host1

```
root@host1:/# brctl addbr br0
root@host1:/# brctl show
bridge name     bridge id               STP enabled     interfaces
br0             8000.000000000000       no
root@host1:/# ip link add eth1.1 type veth peer name eth1.2
root@host1:/# ip a s
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
       valid_lft forever preferred_lft forever
2: eth1: <BROADCAST,NOARP> mtu 1500 qdisc noop state DOWN group defa
    link/ether 1a:ca:ae:d8:5b:6a brd ff:ff:ff:ff:ff:ff
    inet 10.11.0.1/24 scope global eth1
       valid_lft forever preferred_lft forever
4: br0: <BROADCAST,MULTICAST> mtu 1500 qdisc noop state DOWN group d
    link/ether f2:f2:8d:92:0f:0e brd ff:ff:ff:ff:ff:ff
5: eth1.2@eth1.1: <BROADCAST,MULTICAST,M-DOWN> mtu 1500 qdisc noop s
    link/ether b2:fc:4a:c7:e5:61 brd ff:ff:ff:ff:ff:ff
6: eth1.1@eth1.2: <BROADCAST,MULTICAST,M-DOWN> mtu 1500 qdisc noop s
    link/ether ca:47:e7:c2:e1:4f brd ff:ff:ff:ff:ff:ff
23: eth0@if24: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc noqu
    link/ether 02:42:0a:0a:00:02 brd ff:ff:ff:ff:ff:ff link-netnsid
    inet 10.10.0.2/24 brd 10.10.0.255 scope global eth0
       valid_lft forever preferred_lft forever
root@host1:/# brctl addif br0 eth0
root@host1:/# brctl addif br0 eth1.1
root@host1:/# brctl show
bridge name     bridge id               STP enabled     interfaces
br0             8000.02420a0a0002       no              eth0
                                                        eth1.1
```

2.2.2. Налаштувати IP маршрутизацію

```
root@host1:/# ip a a 10.11.0.1/24 dev eth1.2
root@host1:/# ip a s eth1.2
5: eth1.2@eth1.1: <BROADCAST,MULTICAST,M-DOWN> mtu 1500
    link/ether b2:fc:4a:c7:e5:61 brd ff:ff:ff:ff:ff:ff
    inet 10.11.0.1/24 scope global eth1.2
       valid_lft forever preferred_lft forever
```

2.2.3. Виконати пункт 2.2.1 для host2

2.2.4. Налаштувати IP маршрутизацію на host2

```
root@host2:/# ip a a 10.11.0.2/24 dev eth1.2
root@host2:/# ip a s eth1.2
4: eth1.2@eth1.1: <BROADCAST,MULTICAST,M-DOWN> mtu 150
    link/ether 26:3e:33:05:43:55 brd ff:ff:ff:ff:ff:ff
    inet 10.11.0.2/24 scope global eth1.2
       valid_lft forever preferred_lft forever
```

2.2.5. Включаємо інтерфейси eth1.1 та eth1.2 в обох контейнерах

```
root@host1:/# ip link set eth1.1 up
root@host1:/# ip link set eth1.2 up
root@host1:/# ip a s eth1.1
6: eth1.1@eth1.2: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu
    link/ether ca:47:e7:c2:e1:4f brd ff:ff:ff:ff:ff:ff
root@host1:/# ip a s eth1.2
5: eth1.2@eth1.1: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu
    link/ether b2:fc:4a:c7:e5:61 brd ff:ff:ff:ff:ff:ff
    inet 10.11.0.1/24 scope global eth1.2
       valid_lft forever preferred_lft forever
```

2.2.6. Спробуємо пропінгувати 10.11.0.1 з host2

```
root@host2:/# ping 10.11.0.1
PING 10.11.0.1 (10.11.0.1) 56(84) bytes of data.
From 10.10.0.3 icmp_seq=1 Destination Host Unreachable
From 10.10.0.3 icmp_seq=2 Destination Host Unreachable
From 10.10.0.3 icmp_seq=3 Destination Host Unreachable
From 10.10.0.3 icmp_seq=4 Destination Host Unreachable
From 10.10.0.3 icmp_seq=5 Destination Host Unreachable
From 10.10.0.3 icmp_seq=6 Destination Host Unreachable
```

2.2.7. Пінг відсутній, бо треба ще включити бріджи в обох контейнерах

```
root@host2:/# ip link set br0 up
root@host2:/# ip a s br0
2: br0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500
    link/ether 02:42:0a:0a:00:03 brd ff:ff:ff:ff:f
root@host2:/#
```

Завдання 2.3. Заблокуємо пінгування на host1

2.3.1. Спробуємо знайти потрібну опцію

```
root@host1:/proc/sys/net/ipv4# sysctl -a | grep icmp
net.ipv4.icmp_echo_enable_probe = 0
net.ipv4.icmp_echo_ignore_all = 0
net.ipv4.icmp_echo_ignore_broadcasts = 1
net.ipv4.icmp_errors_use_inbound_ifaddr = 0
net.ipv4.icmp_ignore_bogus_error_responses = 1
net.ipv4.icmp_ratelimit = 1000
net.ipv4.icmp_ratemask = 6168
net.ipv6.icmp.echo_ignore_all = 0
net.ipv6.icmp.echo_ignore_anycast = 0
net.ipv6.icmp.echo_ignore_multicast = 0
net.ipv6.icmp.ratelimit = 1000
net.ipv6.icmp.ratemask = 0-1,3-127
net.netfilter.nf_conntrack_icmp_timeout = 30
net.netfilter.nf_conntrack_icmpv6_timeout = 30
```

2.3.2. Спробуємо пропінгувати з host2

```
root@host2:/# ping -c 3 10.11.0.1
PING 10.11.0.1 (10.11.0.1) 56(84) bytes of data.
64 bytes from 10.11.0.1: icmp_seq=1 ttl=64 time=0.144 ms
64 bytes from 10.11.0.1: icmp_seq=2 ttl=64 time=0.105 ms
64 bytes from 10.11.0.1: icmp_seq=3 ttl=64 time=0.128 ms

── 10.11.0.1 ping statistics ──
3 packets transmitted, 3 received, 0% packet loss, time 2027ms
rtt min/avg/max/mdev = 0.105/0.125/0.144/0.016 ms
```

2.3.3. Перевіримо значення та встановимо потрібне

```
root@host1:/proc/sys/net/ipv4# sysctl net.ipv4.icmp_echo_ignore_all
net.ipv4.icmp_echo_ignore_all = 0
root@host1:/proc/sys/net/ipv4# sysctl net.ipv4.icmp_echo_ignore_all=1
net.ipv4.icmp_echo_ignore_all = 1
root@host1:/proc/sys/net/ipv4# sysctl net.ipv4.icmp_echo_ignore_all
```

2.3.4. Спробуємо пропінгувати знову

```
root@host2:/# ping -c 3 10.11.0.1
PING 10.11.0.1 (10.11.0.1) 56(84) bytes of data.

── 10.11.0.1 ping statistics ──
3 packets transmitted, 0 received, 100% packet loss, time 2051ms
```

Завдання 2.4. Підіймемо простий Web-сервер та зробимо запит з використанням netcat

2.4.1. Запускаємо сервер

```
root@host1:/tmp# cat index.html
 <!DOCTYPE html>
<html>
<body>

<h1>My First Heading</h1>
<p>My first paragraph.</p>

</body>
</html>
root@host1:/tmp# cat server.sh
#!/bin/bash
while true; do { \
  echo -ne "HTTP/1.0 200 OK\r\nContent-Length: $(wc -c <index.html)\r\n\r\n"; \
  cat index.html; } | nc -l -p 8080 ; \
done
root@host1:/tmp# chmod +x server.sh
root@host1:/tmp# ./server.sh
```

2.4.2. Переглянемо зайняті порти

```
root@host1:/tmp# netstat -lpnut
Active Internet connections (only servers)
Proto Recv-Q Send-Q Local Address           Foreign Address         State       PID/Program name
tcp        0      0 127.0.0.11:46043        0.0.0.0:*               LISTEN      -
tcp        0      0 0.0.0.0:8080            0.0.0.0:*               LISTEN      111/nc
udp        0      0 127.0.0.11:49902        0.0.0.0:*                           -
```

2.4.3. Робимо запит з host2 та отримуємо відповідь

```
root@host2:/# nc 10.11.0.1 8080
HTTP/1.0 200 OK
Content-Length: 103

 <!DOCTYPE html>
<html>
<body>

<h1>My First Heading</h1>
<p>My first paragraph.</p>

</body>
</html>
^C
root@host2:/#
```

Додаткові завдання на лабораторну роботу

Завдання 2.5. Сконфігурувати eth1.1 та eth1.2 з використанням файлу interfaces

Завдання 2.6. Дослідити інші випадки використання утиліт

Завдання 2.7. Дослідити використання netns (network netspaces)

Контрольні запитання

1) Які 3и утиліти найважливіші та цікавіші на вашу думку?

2) Як може використовуватись ICMP протокол?

3) Навіщо було потрібно створювати bridge в контейнерах?

4) Чому неможливо пропінгувати інтерфейс, який знаходиться в bridge?

5) Як виглядає процес налаштування маршрутизації?

Навчальні матеріали та ресурси

1. Жураковський Б.Ю Технології інтернету речей. Навчальний посібник [Електронний ресурс] / Б. Ю. Жураковский, І. О. Зенів // КПІ ім. Ігоря Сікорського. – 2021. – 503 с. Режим доступу до ресурсу: https://ela.kpi.ua/handle/123456789/42078

2. Жураковський Б.Ю Технології створення інтернету речей. Комп’ютерний практикум. Навчальний посібник [Електронний ресурс] / Б. Ю. Жураковский, Н.В. Федорова, Є.В. Гаврилко, І. О. Зенів, // КПІ ім. Ігоря Сікорського. – 2021. – 128 с. Режим доступу до ресурсу: https://ela.kpi.ua/handle/123456789/46169